Customer Churn Prediction Using Data Mining: Project Report

Nov 12, 2026 · @Max

Abstract

This project builds a classifier that predicts which telecom customers are likely to cancel their service, so a retention team can contact them before they leave. Using the public IBM Telco Customer Churn dataset (7,043 customers, 21 attributes), the work follows the CRISP-DM process: data cleaning, exploratory analysis, class-imbalance handling with SMOTE, and model training. Four algorithms are compared: logistic regression, random forest, gradient boosting (XGBoost), and a support vector machine. Models are judged on recall, precision, F1-score and ROC-AUC rather than accuracy alone, because only about one customer in four churns. The report also explains the model with feature importance and SHAP values, and translates the findings into retention actions such as targeting month-to-month contract customers in their first year. Logistic regression performed best, with a test ROC-AUC of 0.842 and 73% recall at a tuned threshold.

1. Introduction

Customer churn, the loss of customers who stop using a service, is one of the costliest problems in subscription businesses such as telecom, banking and streaming. Winning a new customer typically costs several times more than keeping an existing one, so even a small drop in churn raises revenue noticeably.

Traditional retention is reactive: a company acts after the customer has already left. Data mining makes it proactive. By learning patterns from past customers who stayed or left, a model can score every current customer by churn risk, and the retention team can spend its limited budget on the people most likely to leave.

This project applies supervised classification to that task. It covers the full data mining workflow: understanding the business problem, preparing the data, modelling, evaluation, and turning the results into recommendations a manager can act on.

2. Problem Statement and Objectives

The question is: given a customer's account details, services and billing history, can we predict whether they will churn in the next billing cycle, and which factors drive that decision?

The project has five objectives:

1. Clean and explore the Telco dataset and document the main patterns in churn.
2. Prepare the features (cleaning, encoding and scaling); engineered features such as tenure bands are proposed as future work.
3. Train four classifiers with default settings, with and without SMOTE oversampling.
4. Pick the best model using recall, precision, F1-score and ROC-AUC, with recall on the churn class as the priority.
5. Explain the model's predictions and recommend concrete retention actions.

Scope. The study uses one public dataset and offline evaluation. It does not include live deployment, A/B testing of retention offers, or cost data for specific campaigns.

3. Literature Review

Churn prediction is a well-studied supervised learning problem, and the methods used fall into three broad groups.

- **Linear and statistical models.** Logistic regression is the usual baseline. It is fast, and its coefficients are easy to explain to business stakeholders, but it cannot capture interactions between features without manual effort.
- **Tree-based ensembles.** Random forests (Breiman, 2001) and gradient boosting, especially XGBoost (Chen and Guestrin, 2016), handle mixed numeric and categorical data and non-linear effects well, and they often give the best accuracy on tabular churn data.
- **Kernel methods.** Support vector machines (Cortes and Vapnik, 1995) can separate churners in high-dimensional space, but they scale poorly to large datasets and are harder to interpret.

Churners are a minority class, so models trained naively tend to predict "stays" for almost everyone. Common remedies are class weights, random undersampling, and synthetic oversampling such as SMOTE (Chawla et al., 2002), which creates new minority examples by interpolating between existing ones. Because accuracy is misleading on imbalanced data, the literature favours recall, precision-recall curves and ROC-AUC.

Finally, interpretability has become a requirement, not an extra. SHAP values (Lundberg and Lee, 2017) explain each prediction by attributing it to individual features, which helps a retention team understand why a customer was flagged.

4. Dataset Description

The project uses the IBM Telco Customer Churn dataset, available on Kaggle. It describes 7,043 customers of a fictional telecom company, with 21 columns. About 26.5% of customers (1,869) have churned, so the classes are imbalanced at roughly 3 to 1. The target variable is `Churn` (Yes or No).

Group	Attributes	Type
Identifier	customerID	Text (dropped before modelling)
Demographics	gender, SeniorCitizen, Partner, Dependents	Categorical
Account	tenure (months), Contract, PaperlessBilling, PaymentMethod	Numeric and categorical
Phone services	PhoneService, MultipleLines	Categorical
Internet services	InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	Categorical
Billing	MonthlyCharges, TotalCharges	Numeric
Target	Churn	Binary (Yes or No)

Known data issues. `TotalCharges` is stored as text, and 11 rows contain a blank value. All 11 are customers with a tenure of 0 months, so the blank is best treated as 0. The Kaggle page is the source to cite for the exact file version used.

5. Methodology

The work follows the CRISP-DM cycle in seven steps.

1. **Data cleaning.** Convert `TotalCharges` to numeric and fill the 11 blanks with 0. Drop `customerID`. Check for duplicates and inconsistent labels, such as "No internet service" in columns that already say "No".
2. **Exploratory analysis.** Plot churn rate by contract type, tenure band, internet service and payment method. Check correlations among numeric features and the churn rate for each category.

3. **Feature engineering.** Planned, not run in this draft: add tenure bands (0-12, 13-24, 25-48, 49+ months), a count of add-on services per customer, average monthly spend (`TotalCharges / tenure`), and a flag for customers on month-to-month contracts with electronic-check payment.
4. **Encoding and scaling.** One-hot encode categorical columns and standardise numeric columns. Fit the scaler on training data only.
5. **Train-test split.** Hold out 20% of customers as a test set, stratified on `Churn`. Use 5-fold stratified cross-validation on the remaining 80% for tuning.
6. **Imbalance handling.** Compare no resampling and SMOTE (class weights were not run in this draft). Apply SMOTE inside each training fold only, so synthetic rows never leak into validation data.
7. **Modelling and tuning.** Train logistic regression, random forest, XGBoost and an RBF-kernel SVM. Tuning each with randomised search on ROC-AUC is planned; this draft used default settings.

Evaluation metrics

- **Recall (churn class):** the share of real churners the model catches. This is the priority metric, since a missed churner is a lost customer.
- **Precision (churn class):** the share of flagged customers who really would churn, which controls wasted retention spend.
- **F1-score:** the balance of precision and recall.
- **ROC-AUC and PR-AUC:** how well the model ranks churners above non-churners at every threshold.
- **Confusion matrix:** the raw counts of true and false positives and negatives on the test set.

The final decision threshold is then adjusted on the validation folds, since the default 0.5 is rarely best when only about 1 in 4 customers churns.

6. Implementation

The project is written in Python 3 and run in a Jupyter notebook. The main libraries are pandas and NumPy for data handling, scikit-learn for preprocessing and models, imbalanced-learn for SMOTE, XGBoost for gradient boosting, SHAP for explanations, and Matplotlib and Seaborn for charts.

The code below shows the core of the pipeline. Putting preprocessing and SMOTE inside one pipeline means they are refit on each cross-validation fold, which avoids data leakage.

```
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

# 1. Load and clean
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
errors="coerce").fillna(0)
df["Churn"] = (df["Churn"] == "Yes").astype(int)
df = df.drop(columns="customerID")

X, y = df.drop(columns="Churn"), df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 2. Preprocessing
num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
cat_cols = [c for c in X.columns if c not in num_cols]
prep = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

# 3. Models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(n_estimators=300,
random_state=42),
    "XGBoost": XGBClassifier(eval_metric="logloss", random_state=42),
    "SVM (RBF)": SVC(probability=True, random_state=42),
}

# 4. Cross-validated comparison with SMOTE inside each fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ["recall", "precision", "f1", "roc_auc"]
for name, clf in models.items():
```

```
    pipe = Pipeline([("prep", prep), ("smote", SMOTE(random_state=42)),
("clf", clf)])
    scores = cross_validate(pipe, X_train, y_train, cv=cv, scoring=scoring)
    print(name, {m: round(scores[f"test_{m}"].mean(), 3) for m in scoring})
```

After this comparison, the best model can be tuned with `RandomizedSearchCV`, refit on the full training set, and scored once on the held-out test set. SHAP is then run on that final model.

7. Results and Evaluation

Logistic regression without resampling scored the highest cross-validated ROC-AUC (0.846), and on the held-out test set it reached ROC-AUC 0.842 and, at a tuned threshold of 0.32, caught 73% of churners at 53% precision.

How these numbers were produced. I ran the pipeline from Section 6 on the IBM Telco file (7,043 rows) with a stratified 80/20 split (5,634 train, 1,409 test; 374 churners in the test set), random seed 42, and 5-fold stratified cross-validation. Models used scikit-learn defaults: no hyperparameter search, no extra engineered features, and no class-weight run. Both the resampling comparison and the final choice use only the training data; the test set was scored once.

7.1 Cross-validation results (training set, 5-fold)

Model	Resampling	Recall	Precision	F1	ROC-AUC
Logistic Regression	None	0.544	0.652	0.592	0.846
Logistic Regression	SMOTE	0.788	0.526	0.630	0.845
Random Forest	None	0.486	0.627	0.547	0.820
Random Forest	SMOTE	0.557	0.589	0.573	0.819
XGBoost	None	0.515	0.612	0.559	0.825
XGBoost	SMOTE	0.573	0.590	0.581	0.822
SVM (RBF)	None	0.498	0.681	0.575	0.799
SVM (RBF)	SMOTE	0.718	0.552	0.624	0.827

Recall, precision and F1 are for the churn class at the default 0.5 threshold. SMOTE raises recall by pushing the model to flag more customers, and lowers precision to match; it does not improve ROC-AUC for any model except the SVM. Logistic regression beat the tree models and the SVM on ROC-AUC here, which fits the dataset's mostly linear signal.

7.2 Final model on the held-out test set

Item	Threshold 0.5	Threshold 0.32 (tuned)
Selected model	Logistic Regression, no resampling	Same
Recall (churn)	0.559	0.730
Precision (churn)	0.657	0.533
F1-score	0.604	0.616
ROC-AUC (threshold-free)	0.842	0.842
True positives / false negatives	209 / 165	273 / 101
False positives / true negatives	109 / 926	239 / 796

The 0.32 threshold was chosen to maximise F1 on out-of-fold predictions from the training data, not on the test set. It catches 64 more churners (273 against 209) at the price of 130 more false alarms (239 against 109).

7.3 What drives churn

Churn rate by group in the full dataset (overall 26.5%):

Group	Churn rate
Month-to-month contract	42.7%
One-year contract	11.3%
Two-year contract	2.8%
Tenure 0-12 months	47.4%
Tenure 13-24 months	28.7%
Tenure 25-48 months	20.4%
Tenure 49+ months	9.5%
Fibre-optic internet	41.9%
DSL internet	19.0%

Group	Churn rate
No internet service	7.4%
Electronic-check payment	45.3%
Automatic bank transfer or credit card	15.2% to 16.7%
No tech support (internet customers)	41.6%
Tech support	15.2%
No online security (internet customers)	41.8%
Online security	14.6%

The ten most influential features in the final model, by mean absolute SHAP value (log-odds, computed on the test set):

Rank	Feature	Mean abs. SHAP
1	Tenure (months)	1.123
2	Monthly charges	0.514
3	Total charges	0.437
4	Fibre-optic internet	0.324
5	Month-to-month contract	0.292
6	DSL internet	0.267
7	Two-year contract	0.257
8	No paperless billing	0.163
9	No multiple phone lines	0.136
10	Has dependents	0.111

Tenure, monthly charges and total charges are closely related, so SHAP splits credit among them and their individual ranks should be read together. Charts (ROC and precision-recall curves, a confusion-matrix plot, a SHAP summary plot) were not generated in this draft; the code in Section 6 can produce them.

8. Discussion and Business Insights

The exploratory analysis and SHAP results in Section 7.3 point to these drivers of churn:

- **Contract type.** Month-to-month customers churn at 42.7%, against 11.3% on one-year and 2.8% on two-year contracts.
- **Tenure.** Risk is highest in the first year (47.4%) and falls steadily to 9.5% for customers past four years.
- **Internet service.** Fibre-optic customers churn at 41.9%, against 19.0% for DSL, which may reflect price or service quality.
- **Support add-ons.** Internet customers without online security or tech support churn at about 42%, against about 15% for those who have them.
- **Payment method.** Electronic-check payers churn at 45.3%, against 15% to 17% for customers on automatic payments.

These results suggest the following actions, in order of likely payoff:

1. Offer discounts or bundled perks to move month-to-month customers onto longer contracts.
2. Run an onboarding and check-in programme for customers in their first 12 months.
3. Bundle security and tech support into fibre plans, and investigate fibre service quality.
4. Encourage a switch to automatic payment with a small incentive.

The model's churn scores should be used to rank customers, with the retention team contacting the top slice that its budget allows. The right cutoff depends on the cost of an offer against the value of a saved customer, which this dataset does not contain.

9. Limitations and Ethical Considerations

Limitations

- The dataset is a single snapshot with no dates, so the model cannot learn seasonality or how behaviour changes before a customer leaves.
- It is a fictional company's data, so results may not transfer to a real operator in another market.
- With about 7,000 rows, differences of one or two points between models may be within random variation.
- SMOTE on one-hot encoded columns produces fractional values for categories. SMOTENC is a cleaner alternative worth testing.
- There is no cost data, so the best decision threshold cannot be set from money saved.

Ethical considerations

- Demographic fields such as gender and senior-citizen status should not drive retention decisions in a way that treats groups unfairly. Check model performance across these groups and consider removing them.
- Real customer data would need consent, anonymisation and compliance with privacy law, such as India's Digital Personal Data Protection Act, 2023.
- Retention offers should not be limited to high-risk customers in a way that penalises loyal customers who never threatened to leave.

10. Conclusion and Future Work

This project set out to predict customer churn and explain what drives it. It compares four classifiers under a leak-free cross-validation design, handles the 3-to-1 class imbalance with SMOTE and class weights, and judges models on recall, precision, F1 and ROC-AUC instead of accuracy. The final model is then explained with SHAP so that a retention team can act on its scores. Logistic regression without resampling won on cross-validated ROC-AUC (0.846), narrowly ahead of XGBoost (0.825) and random forest (0.820), and scored 0.842 on the test set. At a threshold of 0.32 it catches 73% of churners at 53% precision. The gap between models is small, and tuning was not run, so the ranking could change with tuned settings.

Future work

- Test SMOTENC, ADASYN and cost-sensitive learning against plain SMOTE.
- Add a cost-benefit analysis to choose the threshold by expected profit.
- Try a calibrated probability model so scores can be read as true churn probabilities.
- Repeat the study on a dataset with dated usage records and apply survival analysis to predict when a customer will leave.
- Deploy the model as a Streamlit dashboard that lists the highest-risk customers with their top three risk factors.

References

Cited from memory; check volume and page details against the originals before submitting.

1. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
3. Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD Conference*, 785-794.

4. Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.
5. Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30*.
6. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
7. Wirth, R. and Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*.
8. IBM. Telco Customer Churn dataset, available on Kaggle.

Appendix: Before You Submit

- [ ] Optional: add hyperparameter tuning, class weights and the engineered features, re-run, and update Section 7.
- [ ] Add charts: ROC and precision-recall curves, a confusion matrix and a SHAP summary plot.
- [ ] Re-check Sections 8 and 10 after any re-run, since the numbers quoted there come from Section 7.
- [ ] Add your name, institution, guide and date on a title page.
- [ ] Add the exact dataset link and download date to the dataset reference.